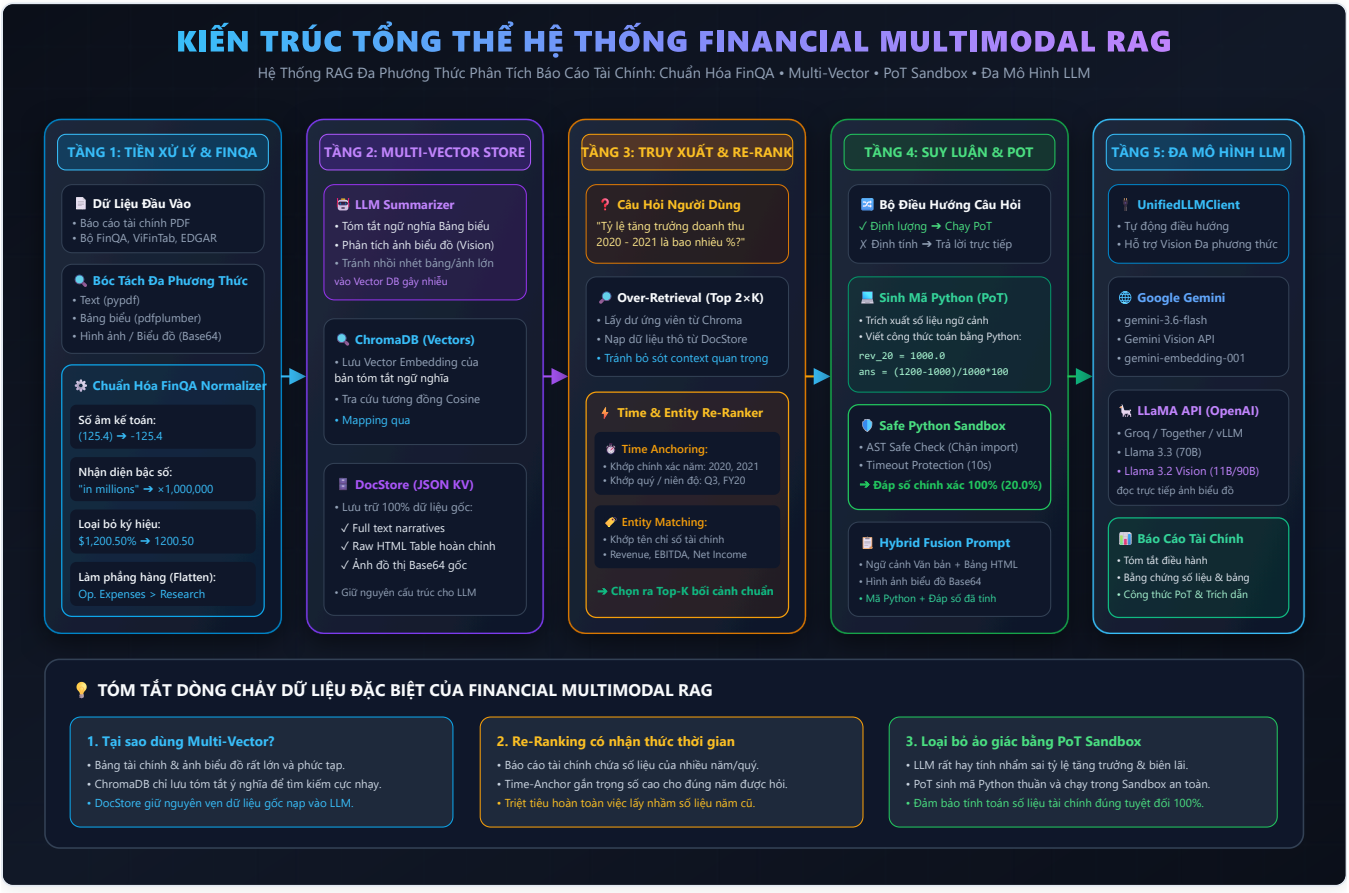
Financial Multimodal RAG — Hybrid Knowledge Graph & Document Tree

Hệ thống **Retrieval-Augmented Generation (GraphRAG) Đa phương thức** thế hệ mới chuyên sâu cho phân tích báo cáo tài chính (Báo cáo thường niên, Báo cáo tài chính Quý/Năm, 10-K, SEC filings, bảng cân đối kế toán, thuyết minh tài chính).

Hệ thống kết hợp sức mạnh đột phá của **Đồ Thị Tri Thức (Knowledge Graph)**, **Cây Phân Cấp Tài Liệu (Document Tree)**, **Vector Retriever (ChromaDB)**, **Chuẩn hóa dữ liệu kế toán FinQA**, **Context Re-ranking Đa Chiều (Time + Entity + Graph Proximity)**, **Program-of-Thought (PoT) với Python Sandbox**, cùng kiến trúc linh hoạt **Đa mô hình LLM (Google Gemini & LLaMA API)**.

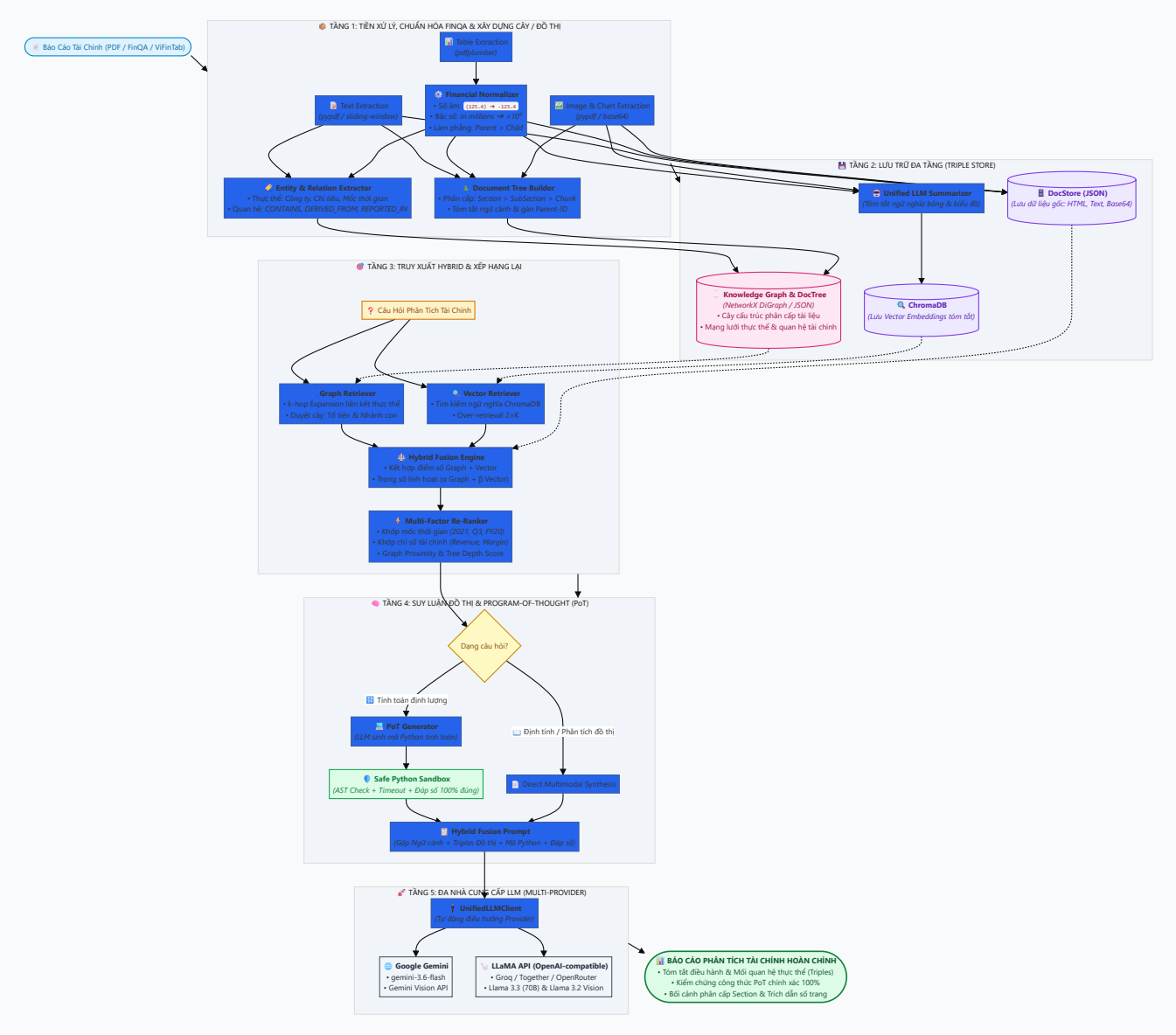
🏛️ Sơ Đồ Kiến Trúc Tổng Thể (System Architecture)

> 📖 **Xem tài liệu kiến trúc chuyên sâu bằng tiếng Việt: 👉 KIEN_TRUC.md**



Sơ Đồ Kiến Trúc Hệ Thống

1. Sơ Đồ Trực Quan Đồ Họa (Interactive Mermaid Diagram)



---

2. Luồng Xử Lý 5 Bước Cốt Lõi (Pipeline at a Glance)

> 🔹 **Bước 1 — Ingestion, Chuẩn Hóa FinQA & Khai Phá Cây/Đồ Thị**: Bóc tách song song Văn bản, Bảng biểu và Đồ thị. Bảng biểu được chuyển sang HTML và chuẩn hóa kế toán (số âm `(x)` → `-x`, quy mô `"in millions"`, làm phẳng `Parent > Child`). Đồng thời, `EntityRelationExtractor` trích xuất các thực thể và quan hệ tài chính, còn `DocumentTreeBuilder` dựng cây phả hệ cấu trúc tài liệu (`Section > SubSection > Chunk`).

> 🔹 **Bước 2 — Lưu Trữ Đa Tầng Triple Store (Knowledge Graph + ChromaDB + DocStore)**:
>
> - **Knowledge Graph (NetworkX)**: Lưu trữ cấu trúc cây tài liệu và mạng lưới thực thể tài chính (`Company` , `Metric` , `Period` , `Value` , `Table` , `Image`) cùng các liên kết (`CONTAINS` , `DERIVED_FROM` , `REPORTED_IN` , `CROSS_REF`).
> - **ChromaDB**: Lưu trữ vector embedding của các bản tóm tắt ngữ nghĩa để tìm kiếm tương đồng vector.
> - **DocStore**: Lưu trữ toàn văn bản gốc, mã bảng HTML đầy đủ và ảnh Base64.

🔹 **Bước 3 — Truy Xuất Kết Hợp Hybrid (Graph Traversal + Vector Search) & Re-ranking Đa Chiều**: Khi người dùng đặt câu hỏi, `GraphRetriever` thực hiện k-hop traversal trên đồ thị tri thức và duyệt cây tài liệu lấy bối cảnh tổ tiên/con cháu, kết hợp cùng `VectorRetriever` tìm kiếm tương đồng ngữ nghĩa. Sau đó, `ContextReRanker` tái xếp hạng dựa trên:

1. **Time Anchoring**: Mốc thời gian (Năm 2021, Quý 3, FY2023).
2. **Financial Entity Matching**: Tên chỉ số (Doanh thu, EBITDA, Biên lợi nhuận).
3. **Graph Proximity Score**: Độ gần gũi thực thể trên đồ thị tri thức.
4. **Tree Depth Score**: Quan hệ phân cấp trong cùng chương mục báo cáo.

🔹 **Bước 4 — Tính Toán Định Lượng Chuẩn Xác với Program-of-Thought (PoT)**: Với các câu hỏi tính toán tỷ lệ tăng trưởng, tỷ suất biên hoặc so sánh đa kỳ, hệ thống tránh hoàn toàn "ảo giác số học" của LLM bằng cách sinh mã Python và thực thi trong **Safe Python Execution Sandbox** (AST validator + timeout) để đảm bảo kết quả chính xác 100%.

🔹 **Bước 5 — Tổng Hợp Báo Cáo Chuyên Sâu với Đa Mô Hình LLM**: `UnifiedLLMClient` kết hợp ngữ cảnh văn bản, bảng biểu, đồ thị tri thức (subgraph triples), cấu trúc cây (document breadcrumbs) cùng đáp số tính toán chuẩn xác để gửi đến **Google Gemini** hoặc **LLaMA API (Groq, Together AI, OpenRouter)** sinh báo cáo phân tích tài chính có trích dẫn chi tiết.

---

3. Bảng Ma Trận Kiến Trúc Hệ Thống (Architecture Matrix)

Tầng (Layer)	Thành Phần Chính	Chức Năng Cốt Lõi	File Nguồn Liên Quan
1. Ingestion, Tree & Graph	`DocumentLoader` `FinancialNormalizer` `EntityRelationExtractor` `DocumentTreeBuilder`	• Trích xuất Text, Table, Chart từ PDF • Chuẩn hóa kế toán FinQA `(125) ➔ -125` • Trích xuất thực thể & quan hệ tài chính • Xây dựng cây phân cấp tài liệu (Document Tree)	`document_loader.py` `financial_normalizer.py` `entity_extractor.py` `tree_builder.py` `models.py`
2. Multi-Tier Triple Store	`KnowledgeGraphManager` `FinancialVectorStore` `DocumentStore` `MultimodalSummarizer`	• Lưu đồ thị tri thức & cây tài liệu qua NetworkX • Lưu Vector Embeddings tóm tắt trong ChromaDB • Lưu dữ liệu thô (HTML/Text/Base64) trong DocStore • Sinh tóm tắt ngữ nghĩa bảng biểu & hình ảnh	`knowledge_graph.py` `vector_store.py` `doc_store.py` `multimodal_summarizer.py`
3. Hybrid Retrieval & Re-ranking	`GraphRetriever` `FinancialVectorStore` `ContextReRanker`	• k-hop Graph Expansion & Tree Ancestor Context • Semantic Vector Search (Over-retrieval 2×K) • Hợp nhất điểm Hybrid (Graph + Vector) • Re-ranking: Time + Entity + Graph Proximity + Tree Depth	`graph_retriever.py` `vector_store.py` `reranker.py`
4. Reasoning & PoT Sandbox	`ProgramOfThoughtEngine` `ExecutionSandbox`	• Tự động nhận diện câu hỏi định lượng tài chính • Sinh Python code tính toán từng bước • AST Validator chặn mã độc + Timeout Protection • Đáp số tính toán chuẩn xác 100%	`pot_reasoning.py` `sandbox.py`
5. Multi-Provider Engine	`UnifiedLLMClient` `FinancialQAEngine`	• Tích hợp **Google Gemini** & **LLaMA API** • Hỗ trợ Vision đọc trực tiếp biểu đồ tài chính • Tích hợp Subgraph Triples & Document Breadcrumbs vào Prompt	`llm_client.py` `financial_qa.py`

🚀 Các Tính Năng Nổi Bật

1. **Đồ Thị Tri Thức Tài Chính (Financial Knowledge Graph) (`knowledge_graph.py`)**:
   - Quản lý mạng lưới thực thể và quan hệ bằng **NetworkX**, được lưu trữ bền vững dưới định dạng Node-Link JSON.
   - Kết nối đa chiều giữa: Doanh nghiệp (`Company`), Chỉ số tài chính (`FinancialMetric`), Kỳ báo cáo (`TimePeriod`), Bảng biểu (`Table`), Biểu đồ (`Chart`), và Đoạn văn (`TextChunk`).
   - Hỗ trợ các liên kết ngữ nghĩa: `CONTAINS`, `DERIVED_FROM` (công thức tính), `REPORTED_IN` (báo cáo trong bảng/kỳ), `CROSS_REF` (thuyết minh liên quan).

2. **Cây Cấu Trúc Phân Cấp Tài Liệu (Document Tree Hierarchy) (`tree_builder.py`)**:
   - Tái tạo cấu trúc cây logic của báo cáo tài chính: `Root ➔ Section ➔ SubSection ➔ Chunk`.
   - Giúp hệ thống không chỉ lấy các đoạn rời rạc mà còn truy xuất được toàn bộ ngữ cảnh cha (Parent Section Summary) và các bảng biểu/thuyết minh phụ trợ cùng cấp.

3. **Truy Xuất Kết Hợp Hybrid (Graph Traversal + Vector Search) (`graph_retriever.py`)**:
   - Kết hợp ưu điểm của cả hai thế giới: Tìm kiếm ngữ nghĩa mở rộng qua Vector và Tìm kiếm cấu trúc chính xác qua Đồ thị tri thức.
   - Thuật toán `k-hop expansion` mở rộng tri thức xung quanh thực thể truy vấn kết hợp công thức cộng gộp trọng số linh hoạt ($\alpha \cdot \text{Score}_{\text{vector}} + \beta \cdot \text{Score}_{\text{graph}}$).

4. **Chuẩn Hóa Dữ Liệu Tài Chính FinQA (`financial_normalizer.py`)**:
   - Tự động chuyển đổi số âm trong kế toán: `(125.4)` $\rightarrow$ `-125.4`.
   - Bóc tách hệ số quy mô: `"in millions"` $\rightarrow$ nhân 10^6, `"in thousands"` $\rightarrow$ nhân 10^3, hỗ trợ tiếng Việt (*"tỷ đồng"*, *"triệu đồng"*).
   - **Làm phẳng tiêu đề hàng (Flattened Row Headers)**: Gán tiền tố nhóm cha vào từng mục con (ví dụ: `Operating Expenses > Research`), loại bỏ nhầm lẫn giữa các chỉ tiêu trùng tên.

5. **Context Re-ranking Nhận Thức Đa Chiều (`reranker.py`)**:
   - Tái chấm điểm dựa trên 4 tiêu chí cốt lõi: Neo mốc thời gian (**Time Anchoring**), Khớp thực thể (**Entity Matching**), Độ gần gũi đồ thị (**Graph Proximity**), và Vị trí cây phân cấp (**Tree Depth**).

6. **Program-of-Thought (PoT) Reasoning & Sandbox (`pot_reasoning.py`, `sandbox.py`)**:
   - Giải quyết triệt để hiện tượng "ảo giác số học". LLM sinh mã Python và thực thi trong Sandbox an toàn có kiểm tra AST để tính toán chính xác 100%.

7. **Kiến Trúc Đa Nhà Cung Cấp LLM (`llm_client.py`)**:
   - Tương thích liền mạch với **Google Gemini** (`gemini-3.6-flash`) và mọi endpoint **OpenAI-compatible** cho LLaMA (Groq, Together AI, OpenRouter, vLLM, Ollama).

🛠️ Cài Đặt & Cấu Hình

1. Cài Đặt Thư Viện

Khuyến nghị Python 3.11 trở lên:

POWERSHELL

```powershell
# Di chuyển vào thư mục dự án
cd d:\Financial-Multimodal-RAG-main\Financial-Multimodal-RAG-main

# Kích hoạt môi trường ảo (nếu có)
.\.venv\Scripts\Activate.ps1

# Cài đặt các gói phụ thuộc (bao gồm networkx cho Knowledge Graph)
pip install -r requirements.txt
pip install -e .
```

2. Cấu Hình File `.env`

Tạo file `.env` từ mẫu `.env.example`:

POWERSHELL

```powershell
copy .env.example .env
```

Cấu hình Đồ Thị Tri Thức & Cây Tài Liệu (Mới):

ENV

```env
# Đường dẫn lưu trữ đồ thị tri thức dạng NetworkX JSON
GRAPH_STORE_PATH=data/processed/knowledge_graph.json

# Bật/tắt trích xuất đồ thị khi nạp tài liệu
ENABLE_GRAPH_EXTRACTION=true

# Số bước nhảy mở rộng lân cận đồ thị (k-hop expansion)
GRAPH_RETRIEVAL_HOPS=2

# Tỷ trọng kết hợp truy xuất: Graph vs Vector
GRAPH_WEIGHT=0.4
VECTOR_WEIGHT=0.6
```

Tùy Chọn A: Sử dụng LLaMA qua API (Groq - Cực nhanh & Miễn phí)

ENV

```env
LLM_PROVIDER=llama
LLAMA_BASE_URL=https://api.groq.com/openai/v1
LLAMA_API_KEY=gsk_your_groq_api_key_here
LLAMA_CHAT_MODEL=llama-3.3-70b-versatile
GEMINI_API_KEY=your_gemini_api_key_here
```

Tùy Chọn B: Sử dụng Google Gemini gốc

```env
LLM_PROVIDER=gemini
GEMINI_API_KEY=your_gemini_api_key_here
EMBEDDING_MODEL=gemini-embedding-001
CHAT_MODEL=gemini-3.6-flash
```

💻 Hướng Dẫn Sử Dụng

1. Giao Diện Dòng Lệnh (CLI)

A. Nạp và xây dựng đồng thời Cây tài liệu, Đồ thị tri thức & Vector embeddings từ PDF:

```powershell
python -m financial_rag.cli ingest data/raw/sample_report.pdf
```

B. Tải bộ dữ liệu chuẩn FinQA từ HuggingFace:

```powershell
python -c "
from datasets import load_dataset; import os
os.makedirs('data/raw/datasets/huggingface/finqa_parquet', exist_ok=True)
for url, split in [
    ('https://huggingface.co/api/datasets/dreamerdeo/finqa/parquet/default/train/0.parquet', 'train'),
    ('https://huggingface.co/api/datasets/dreamerdeo/finqa/parquet/default/validation/0.parquet', 'valida
    ('https://huggingface.co/api/datasets/dreamerdeo/finqa/parquet/default/test/0.parquet', 'test'),
]:
    import urllib.request
    urllib.request.urlretrieve(url, f'data/raw/datasets/huggingface/finqa_parquet/{split}.parquet')
    print(f'Downloaded {split}.parquet')
"
```

C. Nạp tập dữ liệu vào hệ thống (FinQA / ViFinTab / SEC-EDGAR):

```powershell
python -m financial_rag.cli ingest-dataset finqa --split train --sample-size 100
```

D. Đặt câu hỏi phân tích tài chính (Tự động kích hoạt Hybrid Graph-Vector Retrieval):

```powershell
python -m financial_rag.cli ask "Tốc độ tăng trưởng doanh thu từ năm 2020 đến 2021 là bao nhiêu?"
```

E. Ép buộc tính toán số học chính xác qua Program-of-Thought (`--pot`):

```powershell
python -m financial_rag.cli ask "Tính biên lợi nhuận hoạt động trong quý 3?" --pot
```

---

2. REST API (FastAPI)

Khởi động server:

```powershell
python -m financial_rag.cli serve --port 8000
```

Tài liệu Swagger UI tương tác tại: `http://127.0.0.1:8000/docs`.

Gửi câu hỏi phân tích qua `POST /api/v1/ask`:

```json
{
  "question": "What is the percentage growth in net income between 2018 and 2019?",
  "limit": 6,
  "use_pot": true
}
```

Phản hồi API có kèm cấu trúc đồ thị và tính toán:

```json
{
  "answer": "Net income grew by 18.25% from $400M in 2018 to $473M in 2019...",
  "generated_code": "net_income_2018 = 400.0\nnet_income_2019 = 473.0\nans = round((net_income_2019 - net
  "computed_answer": 18.25,
  "pot_success": true,
  "graph_context": [
    {"source": "Net Income", "relation": "REPORTED_IN", "target": "FY2019", "value": 473.0},
    {"source": "Net Income", "relation": "REPORTED_IN", "target": "FY2018", "value": 400.0}
  ],
  "retrieved_tables": [...],
  "retrieved_images": [...]
}
```

🧪 Chạy Kiểm Thử (Tests)

Chạy bộ kiểm thử tự động toàn diện:

```powershell
python -m pytest tests/ -v
```

Các module kiểm thử chính:

- `test_financial_normalizer.py`: Kiểm thử chuẩn hóa số âm, ký hiệu tiền tệ, scale factor, làm phẳng hàng.
- `test_knowledge_graph.py`: Kiểm thử xây dựng đồ thị NetworkX, k-hop expansion, duyệt cây phả hệ tài liệu.
- `test_hybrid_retriever.py`: Kiểm thử hợp nhất kết quả truy xuất giữa Vector Search và Graph Traversal.
- `test_pot_sandbox.py`: Kiểm thử sandbox an toàn, chặn mã độc, PoT arithmetic solver.
- `test_reranker.py`: Kiểm thử trích xuất Time Anchors và Re-ranking đa yếu tố.
- `test_llm_client.py`: Kiểm thử tích hợp đa nhà cung cấp (LLaMA API + Vision + Gemini).

📁 Cấu Trúc Dự Án

TEXT

```
financial-multimodal-rag/
├── src/financial_rag/
│   ├── api/
│   │   └── main.py                  # FastAPI REST API endpoints
│   ├── config/
│   │   └── settings.py              # Cấu hình đa nhà cung cấp & Knowledge Graph
│   ├── ingestion/
│   │   ├── dataset_adapters.py      # Bộ nạp dữ liệu FinQA, ViFinTab, SEC-EDGAR
│   │   ├── document_loader.py       # Trích xuất Text, HTML Tables, và Images từ PDF
│   │   ├── entity_extractor.py      # [MỚI] Trích xuất Thực thể & Quan hệ tài chính
│   │   ├── tree_builder.py          # [MỚI] Xây dựng Cây phân cấp tài liệu (Document Tree)
│   │   ├── financial_normalizer.py  # Chuẩn hóa giá trị tài chính & Flatten Row Headers
│   │   ├── models.py                # Data models mở rộng (KnowledgeEntity, DocumentTreeNode)
│   │   └── ocr.py                   # OCR fallback hook
│   ├── reasoning/
│   │   ├── financial_qa.py          # QA Engine tích hợp Graph Context, Re-ranking & PoT
│   │   ├── llm_client.py            # Unified LLM Client (Gemini & LLaMA API Vision)
│   │   ├── multimodal_summarizer.py # Tóm tắt ngữ nghĩa cho Text, Tables và Images
│   │   ├── pot_reasoning.py         # Program-of-Thought reasoning engine
│   │   └── sandbox.py               # Safe Python Execution Sandbox với AST check
│   ├── retrieval/
│   │   ├── chunking.py              # Chiến lược phân đoạn đa phương thức
│   │   ├── doc_store.py             # Kho lưu trữ dữ liệu thô (JSON KV-store)
│   │   ├── knowledge_graph.py       # [MỚI] Quản lý Đồ thị tri thức & Cây tài liệu (NetworkX)
│   │   ├── graph_retriever.py       # [MỚI] Truy xuất Hybrid (Graph Traversal + Vector Search)
│   │   ├── reranker.py              # Multi-Factor Re-ranker (Time, Entity, Graph Proximity)
│   │   └── vector_store.py          # ChromaDB Multi-Vector Store & Embedding
│   ├── pipeline.py                  # Điều phối toàn diện pipeline Ingest và Query
│   └── cli.py                       # Rich CLI interface (hỗ trợ --pot và graph stats)
├── tests/                           # Unit & Integration test suite
├── requirements.txt                 # Danh mục thư viện phụ thuộc (bao gồm networkx)
├── .env.example                     # Mẫu cấu hình môi trường chuẩn
├── KIEN_TRUC.md                     # Tài liệu kiến trúc chuyên sâu bằng tiếng Việt
└── README.md                        # Tài liệu hướng dẫn & tổng quan hệ thống
```

📄 Bản Quyền (License)

Dự án được phân phối dưới giấy phép **MIT License**.